華中科技大学

课 程 实 验 报 告

课程名称： C 语言程序设计实验

专业班级： 计算机类 2510 班

学　　号： U2025XXXXX

姓　　名： X　X　X

指导教师： 毛 伏 兵

报告日期： 2025.12.1

计算机科学与技术学院

目 录

1 实验 6 指针程序设计实验 1

1.1 程序改错与跟踪调试 1

1.2 程序完善与修改替换 3

1.3 程序设计 112

1.4 小结 27

2 实验 7 结构与联合 29

2.1 表达式求值的程序验证 29

2.2 源程序修改替换 31

2.3 程序设计 35

2.4 小结 57

参考文献 59

1 实验 6 指针程序设计实验

1.1 程序改错与跟踪调试

在下面所给的源程序中，函数 strcopy(t,s)的功能是将字符串 s 复制给字符串 t，并且返回串 t 的首地址。请单步跟踪程序，根据程序运行时出现的现象或观察到的字符串的值，分析并排除源程序的逻辑错误，使之能按照要求输出如下结果：

Input a string:

programming↙ （键盘输入）

programming

Input a string again:

language↙ （键盘输入）

Language

源程序：

```
1.  /* 实验 6-1 程序改错与跟踪题源程序 */
2.  #include<stdio.h>
3.  char * strcopy(char *, const char *);
4.  int main(void)
5.  {
6.      char *s1, *s2, *s3;
7.      printf("Input a string:\n", s2);
8.      scanf("%s", s2);
9.      strcopy(s1, s2);
10.     printf("%s\n", s1);
11.     printf("Input a string again:\n", s2);
12.     scanf("%s", s2);
13.     s3=strcopy(s1, s2);
14.     printf("%s\n", s3);
15.     return 0;
16. }
17.
18. /* 将字符串 s 复制给字符串 t，并且返回串 t 的首地址 */
19. char * strcopy(char * t,const char * s)
20. {
21.     while(* t++= * s++);
22.     return (t);
```

```
23.  }
```

错误修改：

1）第 6 行未为字符指针分配空间，应改为 char *s1=(char*)malloc(50), *s2=(char*)malloc(50) ， *s3=(char*)malloc(50) ， 同时添加头文件 #include<stdlib.h>，最后在程序结束前使用 free 释放分配的内存;

2）第 13 行 s3 不是可修改的左值，正确形式为：strcopy(s3, s2);

3）第 22 行返回的不是字符串 t 的首地址，而是 t 进行若干次++后的地址，

两者差为(串长+1),正确形式为：

```
1.   char* start= t;
2.   while (*t++ = *s++);
3.   return start;
```

4）第 7 行和第 11 行的 printf 语句中格式字符串后面有额外的参数 s2，但格式字符串中没有格式说明符，因此这些 s2 参数是多余的，应该去掉。

修改后的代码：

```
1.   /* 实验 6-1 程序改错与跟踪题源程序 */
2.   #include<stdio.h>
3.   #include<stdlib.h>  // 添加 malloc 头文件
4.
5.   char * strcopy(char *, const char *);
6.
7.   int main(void)
8.   {
9.       // 为字符指针分配空间
10.      char *s1 = (char*)malloc(50), *s2 = (char*)malloc(50),
    *s3 = (char*)malloc(50);
11.
12.      printf("Input a string:\n");  // 移除多余的 s2 参数
13.      scanf("%s", s2);
14.      strcopy(s1, s2);
15.      printf("%s\n", s1);
16.
17.      printf("Input a string again:\n");  // 移除多余的 s2 参
   数
18.      scanf("%s", s2);
19.      strcopy(s3, s2);  // 修正：s3 作为目标字符串
```

```
20.      printf("%s\n", s3);
21.
22.      // 释放内存
23.      free(s1);
24.      free(s2);
25.      free(s3);
26.
27.      return 0;
28.  }
29.
30.  /* 将字符串 s 复制给字符串 t，并且返回串 t 的首地址 */
31.  char * strcopy(char * t, const char * s)
32.  {
33.      char *start = t;  // 保存起始地址
34.      while(*t++ = *s++);
35.      return start;     // 返回起始地址
36.  }
```

运行结果：

```
Input a string:
programming
programming
Input a string again:
language
language
```

图 1-1 程序改错与跟踪调试修改后的程序运行结果示意图

1.2 程序完善与修改替换

(1) 下面程序中函数 strsort 用于对字符串进行升序排序，在主函数中输入 N 个字符串存入通过 malloc 动态分配的存储空间，然后调用 strsort 对这 N 个串按字典序升序排序。

① 请在源程序中的下画线处填写合适的代码来完善该程序。

源程序：

```
1.   /* 实验 6-2 程序完善与修改替换第 (1) 题源程序：字符串升序排序 */
2.   #include<stdio.h>
3.   #include<_______>
4.   #include<string.h>
5.   #define N 4
```

```
/* 对指针数组 s 指向的 size 个字符串进行升序排序 */
void strsort ( char *s[], int size )
{
    _______ temp;
    int i, j;
    for(i=0; i<size-1; i++)
        for(j=0; j<size-i-1; j++)
            if ( _______ )
            {
                temp = s[j];
                _______;
                s[j+1] = temp;
            }
}

int main( )
{
    int i;
    char * s[N], t[50];
    for(i=0; i<N; i++)
    {
        gets(t);
        s[i] = (char *)malloc(strlen(t)+1);
        strcpy(_______);
    }
    strsort(_______);
    for(i=0; i<N; i++) puts(s[i]);
    return 0;
}
```

填空后的程序：

```
/* 实验 6-2 程序完善与修改替换第 (1) 题源程序：字符串升序排序 */
#include<stdio.h>
#include<stdlib.h>  // 添加 malloc 头文件
#include<string.h>
#define N 4
/* 对指针数组 s 指向的 size 个字符串进行升序排序 */
void strsort(char *s[], int size)
{
    char *temp;
    int i, j;
    for(i = 0; i < size - 1; i++)
```

```
12.          for(j = 0; j < size - i - 1; j++)
13.              if(strcmp(s[j], s[j + 1]) > 0)
14.              {
15.                  temp = s[j];
16.                  s[j] = s[j + 1];
17.                  s[j + 1] = temp;
18.              }
19.  }
20.  int main()
21.  {
22.      int i;
23.      char *s[N], t[50];
24.      for(i = 0; i < N; i++)
25.      {
26.          gets(t);
27.          s[i] = (char *)malloc(strlen(t) + 1);
28.          strcpy(s[i], t);
29.      }
30.      strsort(s, N);
31.      for(i = 0; i < N; i++)
32.      {
33.          puts(s[i]);
34.          free(s[i]);
35.      }
36.      return 0;
37.  }
```

运行结果：



图 1-2 程序完善与修改替换（1）①填空后的程序运行结果示意图

② 数组作为函数参数其本质类型是指针。例如，对于形参 char *s[]，编译器将其解释为 char **s，两种写法完全等价。请用二级指针形参重写 strsort 函数，并且在该函数体的任何位置都不允许使用下标引用。

修改后的代码：

```
1.  /* 实验 6-2 程序完善与修改替换第 (1) 题源程序：字符串升序排序 */
2.  #include<stdio.h>
3.  #include<stdlib.h>
4.  #include<string.h>
5.  #define N 4
6.  /* 对指针数组 s 指向的 size 个字符串进行升序排序 - 使用二级指针版本 */
7.  void strsort(char **s, int size)
8.  {
9.      char **p, **q;
10.     char *temp;
11.     int i;
12.
13.     for(i = 0; i < size - 1; i++)
14.     {
15.         p = s;  // p 指向数组起始位置
16.         q = s + 1;  // q 指向 p 的下一个位置
17.
18.         // 内层循环，使用指针遍历
19.         while(q < s + size - i)
20.         {
21.             if(strcmp(*p, *q) > 0)
22.             {
23.
24.                 temp = *p;
25.                 *p = *q;
26.                 *q = temp;
27.             }
28.             p++;
29.             q++;
30.         }
31.     }
32. }
33. int main()
34. {
35.     int i;
36.     char *s[N], t[50];
37.
38.     for(i = 0; i < N; i++)
39.     {
```

```
40.          gets(t);  // 使用 gets 输入字符串
41.          s[i] = (char *)malloc(strlen(t) + 1);
42.          strcpy(s[i], t);  // 将输入的字符串复制到分配的内存
   中
43.      }
44.
45.      strsort(s, N);  // 调用排序函数，s 会自动转换为 char**
46.
47.      printf("排序后的字符串：\n");
48.      for(i = 0; i < N; i++) {
49.          puts(s[i]);
50.          free(s[i]);  // 释放内存
51.      }
52.      return 0;
53.  }
```

运行结果：



图 1-3 程序完善与修改替换（1）②修改后的程序运行结果示意图

(2) 下面程序通过函数指针和菜单选择来调用库函数实现字符串操作：串复制 strcpy、串连接 strcat 或串分解 strtok。

① 请在源程序中的下画线处填写合适的代码来完善该程序，使之能按要求输出下面结果：

1 copy string.

2 connect string.

3 parse string.

4 exit.

input a number (1-4) please!

2 ↵ （键盘输入）

input the first string please!

the more you learn, ↙ （键盘输入）

input the second string please!

the more you get.↙ （键盘输入）

the result is the more you learn,the more you get.

源程序：

```
/* 实验 6-3 程序完善与修改替换第（2）题源程序：通过函数指针实现字符串操作 */
#include<stdio.h>
#include<string.h>
int main(void)
{
    _______;
    char a[80], b[80], *result;
    int choice;
    while(1)
    {
        do
        {
            printf("\t\t1 copy string.\n");
            printf("\t\t2 connect string.\n");
            printf("\t\t3 Parse string.\n");
            printf("\t\t4 exit.\n");
             printf("\t\tinput a number (1-4) please!\n");

            scanf("%d", &choice);
        } while(choice<1 || choice>4);
        switch(choice)
        {
            case 1:  p=strcpy; break;
            case 2:  p=strcat; break;
            case 3:  p=strtok; break;
            case 4:  goto down;
        }
        getchar( );
        printf("input the first string please!\n");
        _______;
        printf("input the second string please!\n");
        _______;
        result = _______ (a,b);
```

```
33.          printf("the result is %s\n",result);
34.      }
35.  down:
36.      return 0;
37.  }
```

填空后的代码：

```
1.   /* 实验 6-3 程序完善与修改替换第 (2) 题源程序：通过函数指针实现字符串操作 */
2.   #include<stdio.h>
3.   #include<string.h>
4.
5.   int main(void)
6.   {
7.       char *(*p)(char *, const char *);  // 声明函数指针
8.       char a[80], b[80], *result;
9.       int choice;
10.
11.      while(1)
12.      {
13.          do
14.          {
15.              printf("\t\t1 copy string.\n");
16.              printf("\t\t2 connect string.\n");
17.              printf("\t\t3 Parse string.\n");
18.              printf("\t\t4 exit.\n");
19.              printf("\t\tinput a number (1-4) please!\n");
20.              scanf("%d", &choice);
21.          } while(choice<1 || choice>4);
22.
23.          switch(choice)
24.          {
25.              case 1:  p=strcpy; break;
26.              case 2:  p=strcat; break;
27.              case 3:  p=strtok; break;
28.              case 4:  goto down;
29.          }
30.
31.          getchar();
32.
33.          printf("input the first string please!\n");
34.          gets(a);
```

```
35.
36.          printf("input the second string please!\n");
37.          gets(b);
38.
39.          result = p(a, b);  // 通过函数指针调用相应函数
40.
41.          printf("the result is %s\n", result);
42.      }
43.
44.  down:
45.      return 0;
46.  }
```

运行结果：

```
                1 copy string.
                2 connect string.
                3 Parse string.
                4 exit.
                input a number (1-4) please!
2
input the first string please!
the more you learn,
input the second string please!
the more you get.
the result is the more you learn,the more you get.
```

图 1-4 程序完善与修改替换（2）①填空后的程序运行结果示意图

② 函数指针的一个用途是用于散转程序，即通过一个转移表（函数指针数组）来实现各分支函数处理，从而省去了大量的 if 语句或 switch 语句。转移表中存放了各个函数的入口地址（函数名），根据条件的设定来查表选择执行相应的函数。请使用转移表而不是 switch 语句重写以上程序。

修改后的代码：

```
1.  /* 实验 6-3 程序完善与修改替换第 (2) 题源程序：使用转移表实现字符串操作 */
2.  #include<stdio.h>
3.  #include<string.h>
4.  void remove_newline(char *str) {
5.      size_t len = strlen(str);
```

```
6.      if(len > 0 && str[len-1] == '\n') {
7.          str[len-1] = '\0';
8.      }
9.  }
10. int main(void)
11. {
12.     // 定义函数指针数组（转移表）
13.     char *(*func_table[])(char *, const char *) = {
14.         strcpy,
15.         strcat,
16.         strtok
17.     };
18.     char a[80], b[80], *result;
19.     int choice;
20.
21.     while(1)
22.     {
23.         do
24.         {
25.             printf("\t\t1 copy string.\n");
26.             printf("\t\t2 connect string.\n");
27.             printf("\t\t3 Parse string.\n");
28.             printf("\t\t4 exit.\n");
29.             printf("\t\tinput a number (1-4) please!\n");

30.             scanf("%d", &choice);
31.         } while(choice < 1 || choice > 4);
32.         if(choice == 4) {
33.             break;
34.         }
35.
36.         // 根据选择获取对应的函数指针
37.         char *(*p)(char *, const char *) = func_table[cho
    ice - 1];
38.         getchar();
39.         printf("input the first string please!\n");
40.         fgets(a, sizeof(a), stdin);
41.         remove_newline(a);
42.
43.         printf("input the second string please!\n");
44.         fgets(b, sizeof(b), stdin);
45.         remove_newline(b);
46.         result = p(a, b);  // 通过转移表调用相应函数
47.         printf("the result is %s\n", result);
```

```
48.         }
49.         return 0;
50.     }
```

运行结果：

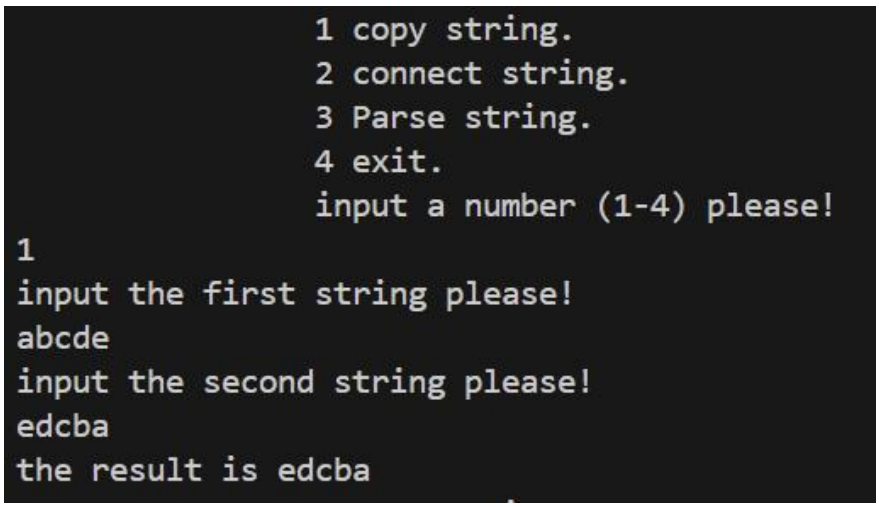


图 1-5 程序完善与修改替换（2）②修改后的程序运行结果示意图

1.3 程序设计

1. **题目**

已知一个长整型变量占 4 字节，其中每字节又分成高 4 位和低 4 位。试编写一个程序，从该长整型变量的高字节开始，依次取出每字节的高 4 位和低 4 位并以十六进制数字字符的形式进行显示，要求通过指针取出每字节。

解题思路：

1. 声明字符指针指向长整型变量的起始地址
2. 将指针移动到高字节位置（起始地址+3）
3. 循环处理 4 个字节：取出当前字节，分离高 4 位和低 4 位，转换为十六进制字符显示
4. 每处理完一个字节，指针向低地址移动

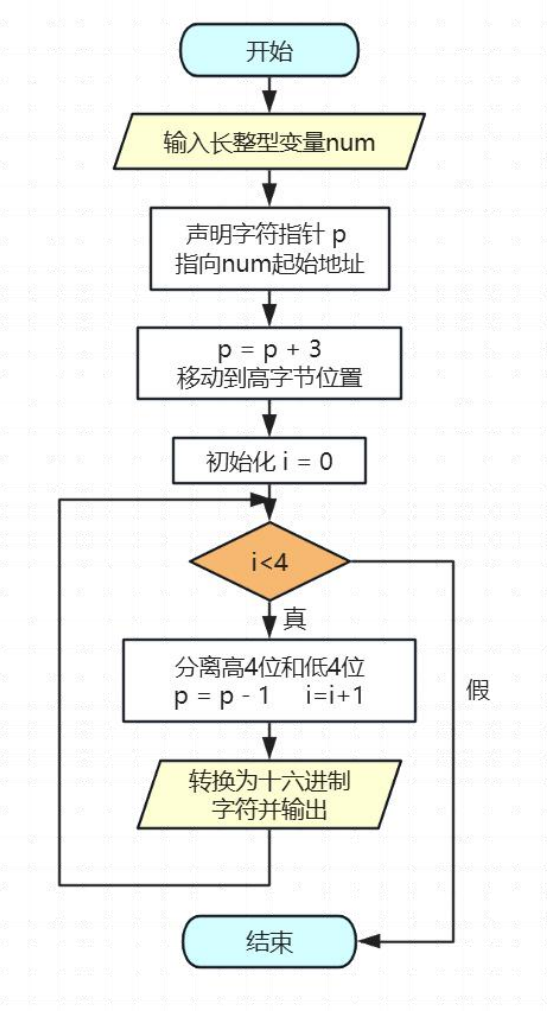


图 1-6 程序设计题 1 的流程图

程序源代码：

```
#include <stdio.h>
char to_hex_char(unsigned char nibble) {
    if (nibble < 10) {
        return '0' + nibble;
    } else {
        return 'A' + (nibble - 10);
    }
}
int main() {
    long num;
    unsigned char *p;
    int i;
    printf("请输入一个长整型数: ");
    scanf("%ld", &num);
    p = (unsigned char *)&num;
```

```
16.       p = p + 3;
17.       printf("长整型数: 0x%lX\n", num);
18.       printf("字节分解结果: ");
19.       for (i = 0; i < 4; i++) {
20.           unsigned char byte = *p;
21.           unsigned char high_nibble = (byte & 0xF0) >> 4;
22.           unsigned char low_nibble = byte & 0x0F;
23.           printf("%c%c ", to_hex_char(high_nibble), to_hex_
   char(low_nibble));
24.           p = p - 1;
25.       }
26.       printf("\n");
27.       return 0;
28.   }
```

运行结果：

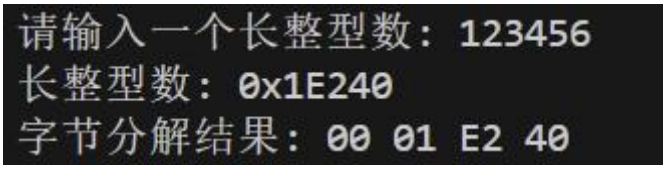


图 1-7 程序设计题 1 的程序运行结果示意图

2. **题目**

定义函数 RemoveSame(a,n)，去掉有 n 个元素的有序整数序列 a 中的重复元素，返回去重后序列的长度值。例如，给定数组 nums = [3,3,3,6,6]，函数调用 RemoveSame(nums,5) 将返回 2，数组 nums 更新为 [3,6,3,6,6]，数组在结果长度后的内容无关紧要。在 main 中给定一个已经排好的数组，删除重复的元素，最后输出去重的结果。要求在程序中均使用指针间接访问数组元素，不能用下标引用。

解题思路：

1. 函数 RemoveSame 的目标是去除有序数组中的重复元素
2. 使用双指针方法：一个指针遍历数组，另一个指针指向不重复元素的位置
3. 当遇到重复元素时，跳过；当遇到不重复元素时，将其复制到前面
4. 返回去重后的数组长度
5. 在 main 函数中，先调用 RemoveSame 去重，然后使用冒泡排序确保数组

有序，最后输出结果

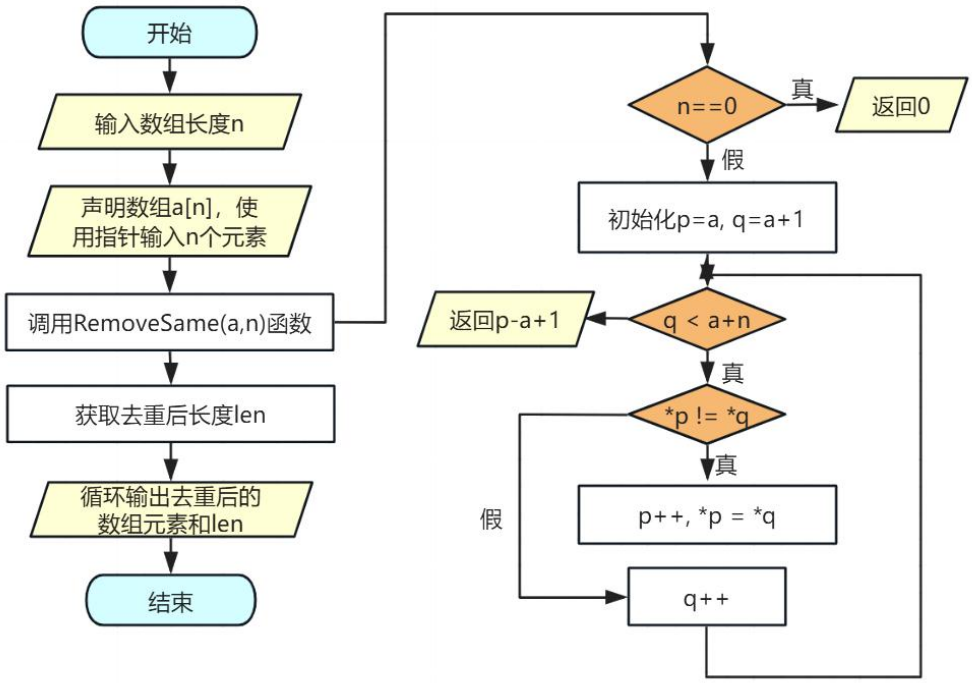


图 1-8 程序设计题 2 的流程图

程序源代码：

```
#include <stdio.h>
int RemoveSame(int *a, int n) {
    if (n == 0) return 0;
    int *p = a;
    int *q = a + 1;
    while (q < a + n) {
        if (*p != *q) {
            p++;
            *p = *q;
        }
        q++;
    }
    return p - a + 1;
}
int main() {
    int n, i;
    scanf("%d", &n);
    int a[n];
```

```
    for (i = 0; i < n; i++) {
        scanf("%d", a + i);
    }
    int len = RemoveSame(a, n);
    for (i = 0; i < len; i++) {
        printf("%d", *(a + i));
        if (i != len - 1) printf(" ");
        else printf("\n");
    }
    printf("%d", len);
    return 0;
}
```

运行结果：

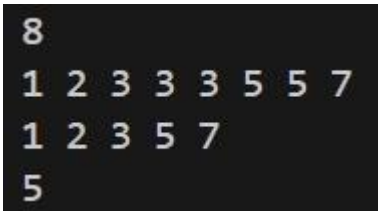


图 1-9 程序设计题 2 的程序运行结果示意图

3. **题目**

旋转是图像处理的基本操作，编程实现将一个图像逆时针旋转 90° 。提示：计算机中的图像可以用一个矩阵来表示，旋转一个图像就是旋转对应的矩阵。将旋转矩阵的功能定义成函数，通过使用指向数组元素的指针作为参数使该函数能处理任意大小的矩阵。要求在 main 函数中输入图像矩阵的行数 n 和列数 m，接下来的 n 行每行输入 m 个整数，表示输入的图像。输出原始矩阵逆时针旋转 90° 后的矩阵。例如，输入：

2 3

1 5 3

3 2 4

则输出

3 4

5 2

1 3

解题思路：

1. 输入矩阵的行数 n 和列数 m
2. 使用二维数组存储矩阵元素
3. 通过指针访问二维数组元素
4. 按列逆序输出矩阵：外层循环控制列号，从最后一列到第一列， 内层循环控制行号，从上到下遍历每一列， 同一列的元素用空格分隔，每列结束后换行

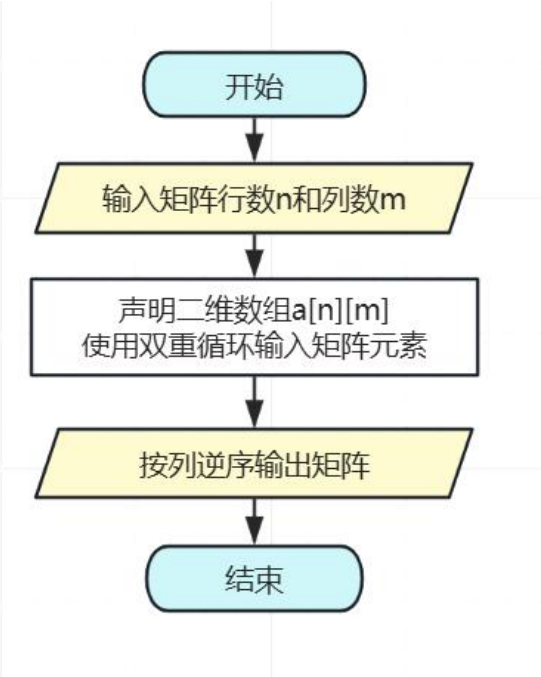


图 1-10 程序设计题 3 的流程图

程序源代码：

```
#include<stdio.h>
int main(){
    int n,m,i,j;
    scanf("%d %d",&n,&m);
    int a[n][m];
    for(i=0;i<n;i++){
        for(j=0;j<m;j++){
            scanf(" %d",(*(a+i)+j));
        }
    }
    for(j=m-1;j>=0;j--){
        for(i=0;i<n;i++){
            printf("%d",*(*(a+i)+j));
            if(i!=n-1)printf(" ");
            else printf("\n");
        }
    }
```

```
18.      return 0;
19.  }
```

运行结果：

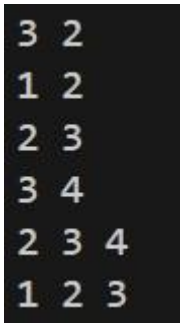


图 1-11 程序设计题 3 的程序运行结果示意图

4. **题目**

编程实现对 n 个整数排序，排序元素的个数 n 由命令行参数指定，排序的原则由命令行可选参数决定，可选参数以负号开头，有参数 -d 时按递减顺序排序，否则按递增顺序排序。要求将排序算法定义成函数。假设该程序名为 sort，那么命令“sort 10 -d”表示对 10 个数按降序排序，在 main 函数中输入 10 个整数和输出排序后的结果，要求 n 个整数的存储无冗余。

解题思路：

1. 程序通过命令行参数获取排序元素个数 n 和排序方式
2. 使用 main 函数的参数 argc 和 argv 处理命令行输入
3. 根据 argc 的值判断是否有-d 参数：
   - 如果 argc==3，表示有-d 参数，按降序排序
   - 如果 argc==2，表示没有-d 参数，按升序排序
4. 使用冒泡排序算法对输入的 n 个整数进行排序
5. 排序完成后输出结果

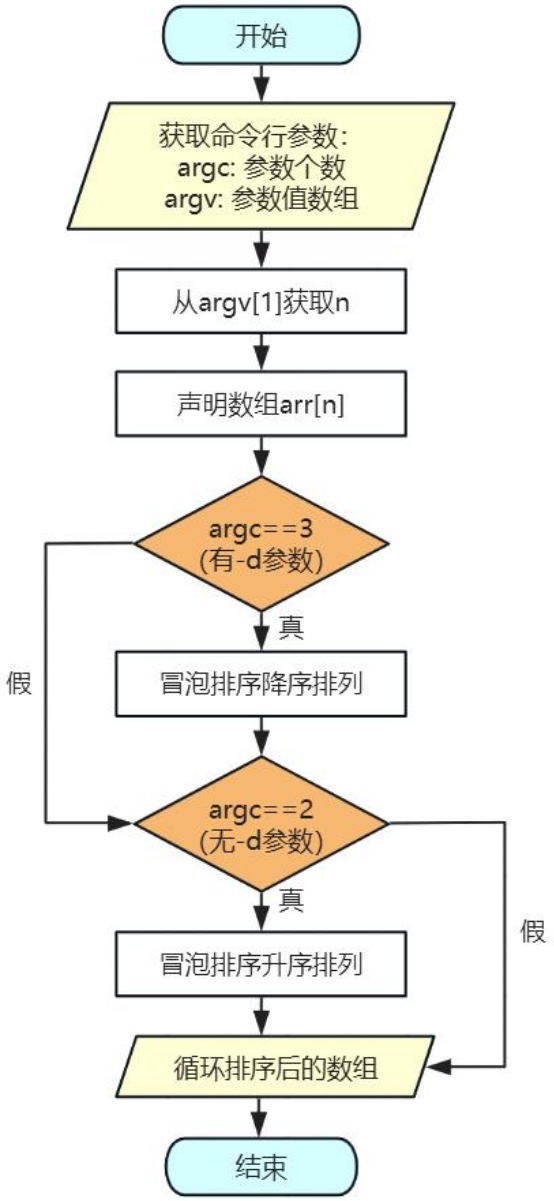


图 1-12 程序设计题 4 的流程图

程序源代码：

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc,char*argv[]){
    int n=atoi(argv[1]);
    int arr[n];
    int i,m;
    for(i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
```

```
    if(argc==3){
      for(i=0;i<n-1;i++){
        for(int j=0;j<n-1-i;j++){
            if(arr[j]<arr[j+1]){
                m=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=m;
            }
        }
    }
    }
    if(argc==2){
      for(i=0;i<n-1;i++){
        for(int j=0;j<n-1-i;j++){
            if(arr[j]>arr[j+1]){
                m=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=m;
            }
        }
    }
    }
    for(i=0;i<n;i++){
        printf("%d",arr[i]);
        if(i!=n-1)printf(" ");
    }
    return 0;
}
```

运行结果：



图 1-13 程序设计题 4 的程序运行结果示意图

5. 题目

编写一个函数 delSubstr(str, substr)，删除字符串 str 中出现的所有子串 substr。如果成功删除子串函数，则返回 1；如果 str 中不包含子串 substr，则 str 未作修改，函数返回 0。要求：子串在函数中不会被修改；函数中的任何地方都不使用下标引用；允许使用 strstr、strcpy 等标准库函数。在 main 函数中输出删除子串后的结果字符串。

解题思路：

1. 输入两个字符串 arr1 和 arr2
2. 使用 strstr 函数在 arr1 中查找 arr2 子串
3. 如果找到，将 arr1 中不包含 arr2 的部分复制到 arr3
4. 统计是否找到 arr2（count=1 表示找到，0 表示未找到）
5. 输出处理后的字符串和查找结果

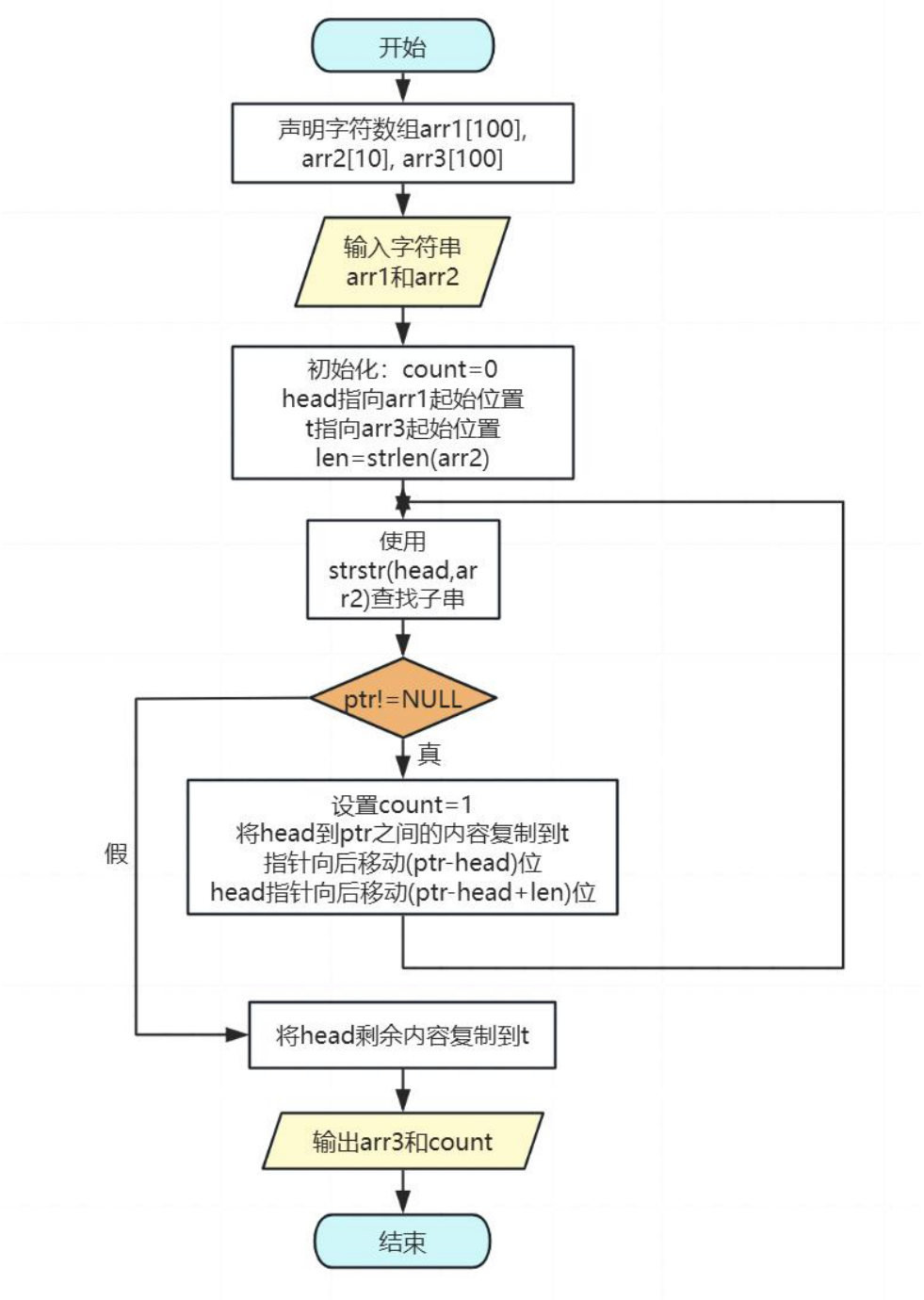


图 1-14 程序设计题 5 的流程图

程序源代码：

```c
#include<stdio.h>
#include<string.h>
int main(){
char arr1[100],arr2[10],arr3[100];
gets(arr1);
gets(arr2);
int count=0;
char*ptr,*head=arr1,*t=arr3;
int len=strlen(arr2);
    while ((ptr = strstr(head,arr2))!=NULL) {

        count=1;
        strncpy(t,head,ptr-head);

        t+=ptr-head;
        head +=ptr-head+len;
    }
    strcpy(t, head);

printf("%s\n%d",arr3,count);
return 0;
}
```

运行结果：

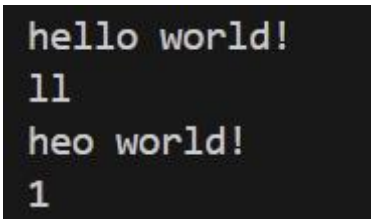


图 1-15 程序设计题 5 的程序运行结果示意图

6. **题目**

求两个不超过 200 位的非负整数的积，输入两个不超过 200 位的非负整数，输出相乘后的结果，结果里不输出多余的前导 0。

解题思路：

1. 大整数存储方式：使用数组逆序存储，数组第一个元素存储数字位数

2. 主要函数： getBigNum：读取大整数并逆序存储， chengBigNum：实现大整数乘法，putBigNum：输出大整数

3. 算法原理：模拟手算乘法，逐位相乘并处理进位

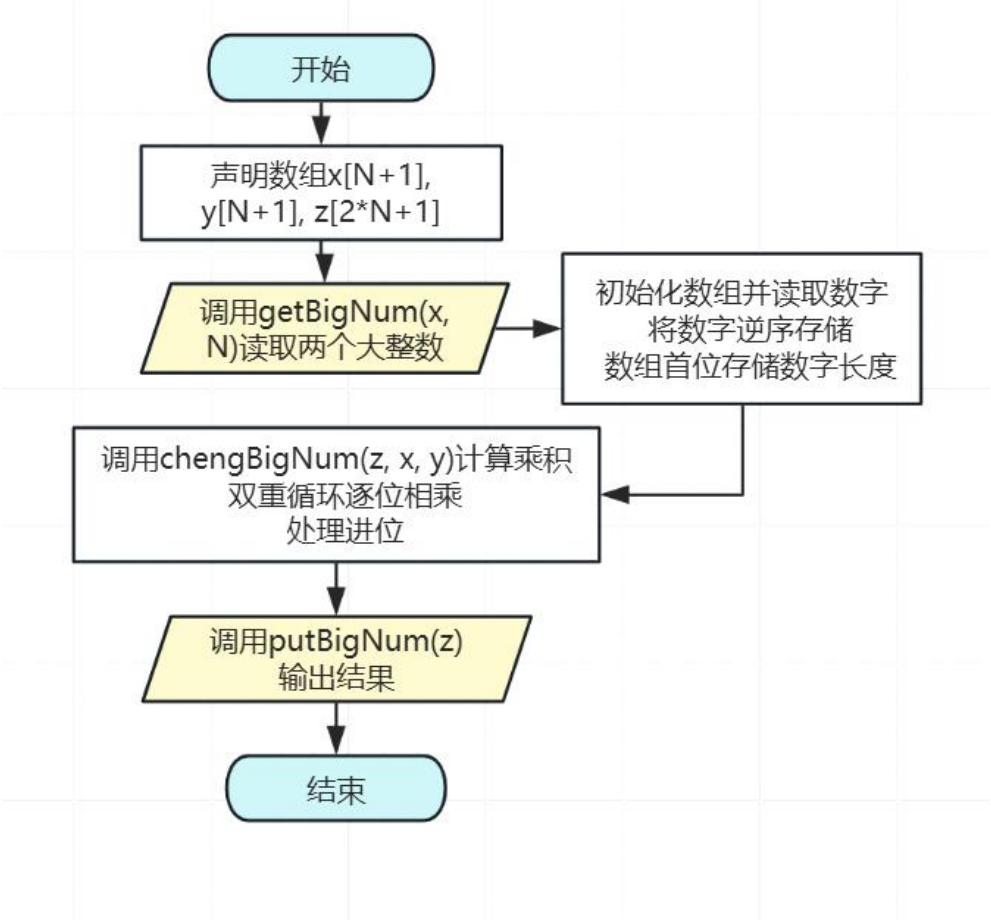


图 1-16 程序设计题 6 的流程图

程序源代码：

```
1.   #include <stdio.h>
2.   #include <ctype.h>
3.   #define N 200
4.   void getBigNum(int *x, int lim) {
5.       int i, t, c;
6.       int *p1, *p2;
7.       for (i = 1; i <= lim; i++)  *(x + i) = 0;
8.
9.       for (i = 1; i <= lim && isdigit(c = getchar()); i++)

10.          *(x + i) = c - '0';
11.      *x = i - 1;
12.      for (p1 = x + 1, p2 = x + (*x); p1 < p2; p1++, p2--)
   {
13.          t = *p1;
14.          *p1 = *p2;
15.          *p2 = t;
16.      }
```

```
}
void chengBigNum(int *z, int *x, int *y) {
    int i, j;
    for (i = 1; i <= 2*N; i++)  *(z + i) = 0;
    for (i = 1; i <= *x; i++) {
        for (j = 1; j <= *y; j++) {
            *(z + i + j - 1) += *(x + i) * (*(y + j));
        }
    }
    for (i = 1; i <= 2*N; i++) {
        if (*(z + i) >= 10) {
            *(z + i + 1) += *(z + i) / 10;
            *(z + i) %= 10;
        }
    }

    i = 2*N;
    while (i >= 1 && z[i] == 0)  i--;
    *z = (i == 0) ? 1 : i;
}

void putBigNum(int *x) {
    int *p;
    int n = *x;
    for (p = x + n; p >= x + 1; p--)
        printf("%d", *p);
}

int main(void) {
    int x[N+1], y[N+1], z[2*N + 1];
    getBigNum(x, N);
    getBigNum(y, N);
    chengBigNum(z, x, y);
    putBigNum(z);
    return 0;
}
```

运行结果：

```
6666666666
2222222222
14814814811851851852
```

图 1-17 程序设计题 6 的程序运行结果示意图

7. **题目**

编写 8 个任务函数，一个 scheduler 调度函数和一个 execute 执行函数。仅在 main 函数中调用 scheduler 函数，scheduler 函数要求用最快的方式调度执行用户指定的任务函数。

① 先设计 task0、task1、task2、task3、task4、task5、task6、task7 共 8 个任务函数，每个任务函数的任务就是输出该任务被调用的字符串。例如，第 0 个任务函数输出“task0 is called!”，第 1 个任务函数输出“task1 is called!”，以此类推。

② scheduler 函数根据键盘输入的数字字符的先后顺序，依次调度选择对应的任务函数。例如，输入：1350 并回车，则 scheduler 函数依次调度选择 task1、task3、task5、task0，然后以函数指针数组和任务个数为参数将调度选择结果传递给 execute 函数并调用 execute 函数。

③ execute 函数根据 scheduler 函数传递的指针数组和任务个数为参数，按照指定的先后顺序依次调用执行选定的任务函数。

例如，当输入 13607122 并回车，程序运行结果如下：

task1 is called!

task3 is called!

task6 is called!

task0 is called!

task7 is called!

task1 is called!

task2 is called!

task2 is called!

解题思路：

1. 定义 8 个任务函数，每个函数输出对应的调用信息
2. scheduler 函数读取输入的数字字符串，将对应任务函数指针存入数组
3. execute 函数根据传入的函数指针数组依次执行任务
4. 使用函数指针数组实现快速调度

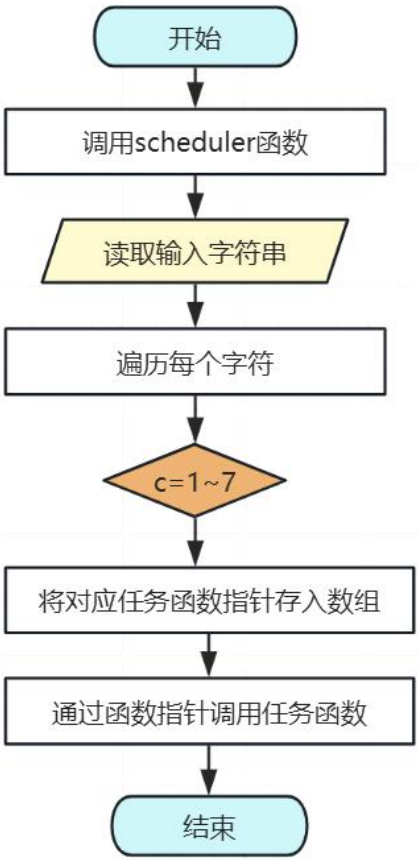


图 1-18 程序设计题 7 的流程图

程序源代码：

```
#include <stdio.h>
#include <string.h>
void task0() { printf("task0 is called!\n"); }
void task1() { printf("task1 is called!\n"); }
void task2() { printf("task2 is called!\n"); }
void task3() { printf("task3 is called!\n"); }
void task4() { printf("task4 is called!\n"); }
void task5() { printf("task5 is called!\n"); }
void task6() { printf("task6 is called!\n"); }
void task7() { printf("task7 is called!\n"); }
void execute(void (**tasks)(void), int count) {
    for (int i = 0; i < count; i++) {
        tasks[i]();
    }
}
void scheduler() {
    char input[100];
    void (*task_ptrs[100])(void);
```

```
    int count = 0;
    fgets(input, sizeof(input), stdin);
    for (int i = 0; input[i] != '\0' && input[i] != '\n'; i++) {
        char c = input[i];
        if (c >= '0' && c <= '7') {
            int task_num = c - '0';
            switch(task_num) {
                case 0: task_ptrs[count] = task0; break;
                case 1: task_ptrs[count] = task1; break;
                case 2: task_ptrs[count] = task2; break;
                case 3: task_ptrs[count] = task3; break;
                case 4: task_ptrs[count] = task4; break;
                case 5: task_ptrs[count] = task5; break;
                case 6: task_ptrs[count] = task6; break;
                case 7: task_ptrs[count] = task7; break;
            }
            count++;
        }
    }
    execute(task_ptrs, count);
}
int main() {
    scheduler();
    return 0;
}
```

运行结果：

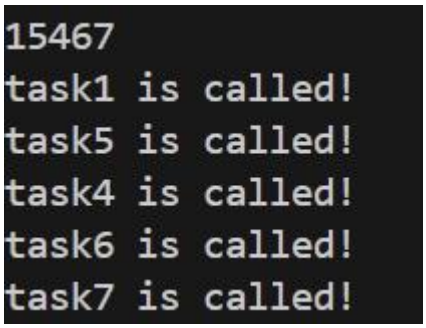


图 1-19 程序设计题 7 的程序运行结果示意图

1.4 小结

在这次指针实验中，我学习了指针的声明、赋值和使用方法，掌握了用指针

引用数组元素的技巧，熟悉了指向数组的指针的使用。通过实践，我熟练掌握了字符数组与字符串的操作，理解了指针数组及字符指针数组的具体应用。以下是我在实验过程中的一些发现和心得体会。

1. 输入函数的选择问题

在实验过程中，我发现不同的输入函数会产生截然不同的效果。例如，scanf 函数在遇到空格时就会停止读取，gets_s 函数遇到回车才停止且不读取回车符，而 fgets_s 函数则会读取回车符。这些差异让我意识到，在输入字符串时需要根据具体需求谨慎选择输入函数。特别是在连续输入多个字符串时，必须注意用 getchar()清除缓冲区，避免残留字符被意外读取，这一点在调试过程中给了我深刻的教训。

2. 函数指针数组的妙用

通过实现任务调度功能，我深刻体会到函数指针数组的强大之处。使用函数指针数组可以大大简化代码结构，提高程序的灵活性和可维护性。当需要根据不同的条件执行不同的函数时，只需要通过索引访问函数指针数组即可，这种设计模式让代码更加优雅简洁。这一点的学习不仅增强了我的编程能力，更让我对指针的本质有了更深层次的理解。

3. 缓冲区与越界问题的注意事项

在实验过程中，我特别注意到缓冲区和越界问题的重要性。使用指针和数组时，必须时刻关注数组的大小边界，避免发生越界访问。同时，在声明和使用指针时，类型匹配是关键细节，只有确保类型的正确性，才能达到预期的运行效果。这些经验让我认识到，严谨细致的态度在编程工作中至关重要。

2 实验 7 结构与联合

2.1 表达式求值的程序验证

设有下面说明，请先自己计算表 7-1 中表达式的值，然后通过编程计算来加以验证。各表达式之间相互无关。

```
1.   char u[] = "UVWXYZ", v[] = "xyz";
2.   struct T {
3.       int x;
4.       char c;
5.       char *t;
6.   } a[] = { { 11, 'A', u }, { 100, 'B', v } }, *p = a;
```

序号	表达式	计算值	验证值
1	(++p)->x	100	100
2	p++->c	B	B
3	p++->t, *p->t	x	x
4	*(++p)->t	x	x
5	++p->t	V	V
6	++*p->t	V	V

计算过程：

1) ++p 先进行 p+1 再运算，此时 p 指向数组第二个元素，原表达式即数组第二个结构中 x 的值 100

2) 逗号表达式从左往右计算，结果取右值，为 B

3) *p++->t：这是一个后缀递增运算符，首先取 p->t 的值，然后递增 p。接着，通过 * 取得该值的内容 U。*p->t：这个表达式首先会取 p->t 的值，然后通过 * 取得该值的内容。所以最终，p 指向第二个结构体的元素 v(指针)，再使用间接引用运算符 *取值为 x，表达式为右值 x。

4) *(++p)->t：先进行 p+1 再运算，再使用间接引用运算符 *对 p 指向的第二个结构体中的元素 v(指针)进行取值为 x

5) *++p->t：首先，会访问结构体或联合体的成员 t：接着，会对 p->t 的结果

进行递增操作,使指针 t 指向数组 u 第二个元素。注意，这是一个前缀递

增运

算符，所以会在取值之前递增 p->t 最后，通过前两步得到的结果，使用间接引用运算符 * 取得其内容。

6）->的优先级大于++和*，而++和*是右结合，综合起来，++*p->t 的操作顺序

是先取得 p->t 的值，然后使用间接引用运算符 * 取得其内容。最后对该内容进行++，此时数组 u 中第一个元素的值变成 V，表达式的值即该值：V

程序源代码：

```
#include <stdio.h>
#include <string.h>
int main() {
    char u[] = "UVMXYZ";
    char v[] = "xyz";
    struct T {
        int x;
        char c;
        char* t;
    } a[] = { {11, 'A', u}, {100, 'B', v} }, *p = a;
    printf("%d\n", (++p)->x);
    p--;
    printf("%c\n", (p++, p->c));
    p--;
    printf("%c\n", (*p++->t, *p->t));
    p--;
    printf("%c\n", *(++p)->t);
    p--;
    printf("%c\n", *++p->t);
    --p->t;
    strcpy(u, "UVMXYZ");
    printf("%c\n", ++*p->t);
    return 0;
}
```

运行结果：



图 2-1 表达式求值的程序运行结果示意图

2.2 源程序修改替换

给定一批整数，以 0 作为结束标志且不作为结点，将其建成一个先进先出的链表，先进链表的头指针始终指向最后创建的结点（链头），先进结点指向后建结点，后建结点始终是尾结点。

（1）源程序中存在什么样的错误（先观察执行结果）？对程序进行修改、调试，使之能够正确完成指定任务。

```
1.  /*实验 7-1 修改替换题：创建先进先出链表*/
2.  #include<stdio.h>
3.  #include<stdlib.h>
4.  struct s_list {
5.     int data;
6.     struct s_list *next;
7.  };
8.  void create_list(struct s_list *headp, int *p);
9.  int main(void)
10. {
11.    struct s_list *head = NULL, *p;
12.    int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 0};
13.    create_list(head, a);
14.    p = head;
15.    while (p) {
16.       printf("%d\t", p->data);
17.       p = p->next;
18.    }
19.    printf("\n");
20.    return 0;
21. void create_list(struct s_list *headp, int *p)
22. {
23.       struct s_list * loc_head=NULL, * tail;
24.    if (p[0] == 0) ;
```

```
    else {
      loc_head = (struct s_list *)malloc(sizeof(struct s_list));
      loc_head->data = *p++;
      tail = loc_head;
      while (*p) {
        tail->next = (struct s_list *)malloc(sizeof(struct s_list))
;
        tail = tail->next;
        tail->data = *p++;
      }
      tail->next = NULL;
    }
    headp = loc_head;
  }
```

错误修改：

1. 参数传递错误：create_list 函数试图修改 head 指针，但 C 语言是值传递，函数内部对 headp 的修改不会影响外部的 head。

2. 内存泄漏：如果输入数组为空，函数没有正确处理。

修改后的代码：

```c
/* 实验 7-1 修改替换题：创建先进先出链表 */
#include<stdio.h>
#include<stdlib.h>
struct s_list {
    int data;           /* 数据域 */
    struct s_list *next; /* 指针域 */
};
void create_list(struct s_list **headp, int *p);
int main(void)
{
    struct s_list *head = NULL, *p;
    int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 0};
    create_list(&head, a);
    p = head; /* 遍历指针 p 指向链头 */
    while (p) {
        printf("%d\t", p->data); /* 输出数据域的值 */
        p = p->next; /* 遍历指针 p 指向下一结点 */
    }
    printf("\n");

    // 释放链表内存
    p = head;
    while (p) {
```

```
24.          struct s_list *temp = p;
25.          p = p->next;
26.          free(temp);
27.      }
28.      return 0;
29.  }
30.  void create_list(struct s_list **headp, int *p)
31.  {
32.      struct s_list *loc_head = NULL, *tail;
33.
34.      if (p[0] == 0) {
35.          *headp = NULL;
36.          return;
37.      }
38.      loc_head = (struct s_list *)malloc(sizeof(struct s_li
   st));
39.      if (!loc_head) {
40.          printf("内存分配失败\n");
41.          *headp = NULL;
42.          return;
43.      }
44.
45.      loc_head->data = *p++; /* 对数据域赋值 */
46.      loc_head->next = NULL;
47.      tail = loc_head; /* tail 指向第一个结点 */
48.
49.      while (*p) {
50.          /* tail 所指结点的指针域指向动态创建的结点 */
51.          tail->next = (struct s_list *)malloc(sizeof(struc
   t s_list));
52.          if (!tail->next) {
53.              printf("内存分配失败\n");
54.              break;
55.          }
56.          tail = tail->next; /* tail 指向新创建的结点 */
57.          tail->data = *p++;
58.          tail->next = NULL; /* 新结点的 next 设为 NULL */
59.      }
60.
61.      *headp = loc_head;   /* 通过二级指针修改外部头指针 */
62.  }
```

运行结果：



图 2-2 原程序修改替换（1）的程序运行结果示意图

（2）修改 create_list 函数，将其建成一个后进先出的链表，后建结点指向先建结点，先建结点始终是尾结点。

修改后的代码：

```
1.  #include<stdio.h>
2.  #include<stdlib.h>
3.
4.  struct s_list {
5.      int data;
6.      struct s_list *next;
7.  };
8.
9.  void create_list(struct s_list **headp, int *p);
10.
11. int main(void)
12. {
13.     struct s_list *head = NULL, *p;
14.     int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 0};
15.     create_list(&head, a);
16.     p = head;
17.     while (p) {
18.         printf("%d\t", p->data);
19.         p = p->next;
20.     }
21.     printf("\n");
22.
23.     p = head;
24.     while (p) {
25.         struct s_list *temp = p;
26.         p = p->next;
27.         free(temp);
28.     }
29.
30.     return 0;
31. }
32.
33. void create_list(struct s_list **headp, int *p)
34. {
35.     struct s_list *new_node;
36.
37.     if (p[0] == 0) {
```

```
38.             *headp = NULL;
39.             return;
40.         }
41.
42.         *headp = NULL;
43.
44.         while (*p) {
45.             new_node = (struct s_list *)malloc(sizeof(struct
    s_list));
46.             if (!new_node) {
47.                 printf("内存分配失败\n");
48.                 return;
49.             }
50.
51.             new_node->data = *p;
52.             new_node->next = *headp;
53.             *headp = new_node;
54.
55.             p++;
56.         }
57.     }
```

运行结果：

```
8       7       6       5       4       3       2       1
```

图 2-3 原程序修改替换（2）的程序运行结果示意图

2.3 程序设计

1. **题目**

设计一个字段结构 struct bits，它将一个 8 位无符号字节从最低位向最高位声明为 8 个字段，名字依次为 b0，b1，b2，…，b7，并且 b0 的优先级最高。同时，设计 8 个函数，将 8 个函数的名字存入一个函数指针数组 p_fun，为参数，在函数体内输出 bii 的值。如果 b0 为 1，则根据优先级从高到低顺序依次调用函数指针数组 p_fun 中相应元素指向的函数。如果 struct bits 中有多位为 1，则 8 个函数中的第 0 个函数可以设计为：

```
void f0(int b) {
    printf("the function %d is called!\n", b);
}
```

解题思路：

1. 程序通过位域结构体来解析输入整数的每个比特位
2. 使用函数指针数组将每个比特位与对应的处理函数关联
3. 根据输入整数的二进制表示中哪些位被置 1，调用相应的函数
4. 每个函数被调用时，传入的参数是该位的值（0 或 1）

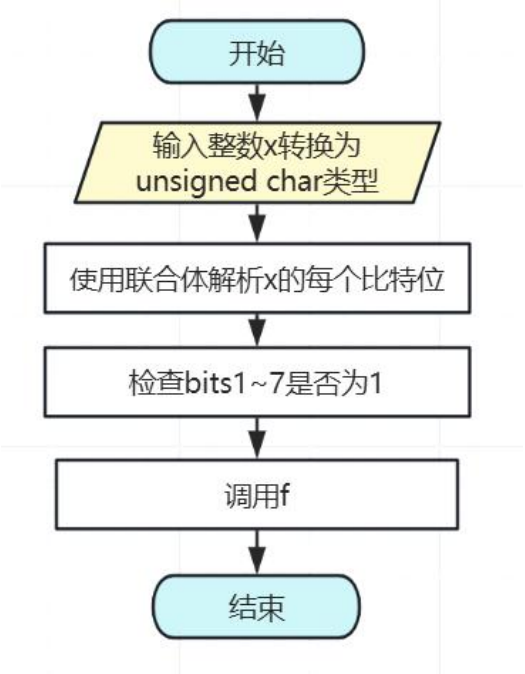


图 2-4 程序设计题 1 的流程图

程序源代码：

```
#include<stdio.h>
struct bits{
    unsigned int bits0:1;
    unsigned int bits1:1;
    unsigned int bits2:1;
    unsigned int bits3:1;
    unsigned int bits4:1;
    unsigned int bits5:1;
    unsigned int bits6:1;
    unsigned int bits7:1;
};
void f0(int b){
    printf("the function %d is called!\n",b-1);
}
void f1(int b){
```

```
16.      printf("the function %d is called!\n",b);
17.  }
18.  void f2(int b){
19.      printf("the function %d is called!\n",b+1);
20.  }
21.  void f3(int b){
22.      printf("the function %d is called!\n",b+2);
23.  }
24.  void f4(int b){
25.      printf("the function %d is called!\n",b+3);
26.  }
27.  void f5(int b){
28.      printf("the function %d is called!\n",b+4);
29.  }
30.  void f6(int b){
31.      printf("the function %d is called!\n",b+5);
32.  }
33.  void f7(int b){
34.      printf("the function %d is called!\n",b+6);
35.  }
36.  int main() {
37.      int x;
38.      scanf("%d",&x);
39.      void (*hanshu[8])(int) = {f0, f1, f2, f3, f4, f5, f6,
    f7};
40.      union {
41.          unsigned char byte;
42.          struct bits bits;
43.      } data;
44.      data.byte = (unsigned char)x;
45.
46.      if (data.bits.bits0) hanshu[0](data.bits.bits0);
47.      if (data.bits.bits1) hanshu[1](data.bits.bits1);
48.      if (data.bits.bits2) hanshu[2](data.bits.bits2);
49.      if (data.bits.bits3) hanshu[3](data.bits.bits3);
50.      if (data.bits.bits4) hanshu[4](data.bits.bits4);
51.      if (data.bits.bits5) hanshu[5](data.bits.bits5);
52.      if (data.bits.bits6) hanshu[6](data.bits.bits6);
53.      if (data.bits.bits7) hanshu[7](data.bits.bits7);
54.
55.      return 0;
56.  }
```

运行结果：

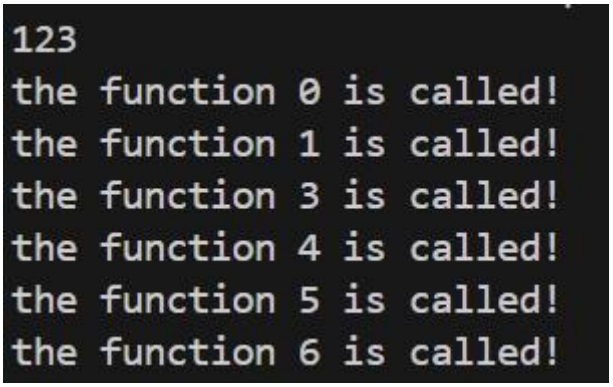


图 2-5 程序设计题 1 的程序运行结果示意图

2. **题目**

用单向链表建立一张班级成绩单，包括每个学生的学号、姓名、英语、高等数学、普通物理，用程序设计 4 门课程的成绩。用函数实现下列功能，并提供菜单选项：

① 输入每个学生的各项信息。

② 输出每个学生的各项信息。

③ 修改指定学生的指定字段的内容。

④ 统计每个学生的平均成绩（保留 2 位小数）。

⑤ 输出各位学生的学号、姓名、4 门课程的总成绩和平均成绩。

解题思路：

1. 使用单向链表存储学生信息

2.每个节点包含：学号、姓名、4 门课程成绩

3.通过指针连接各个节点

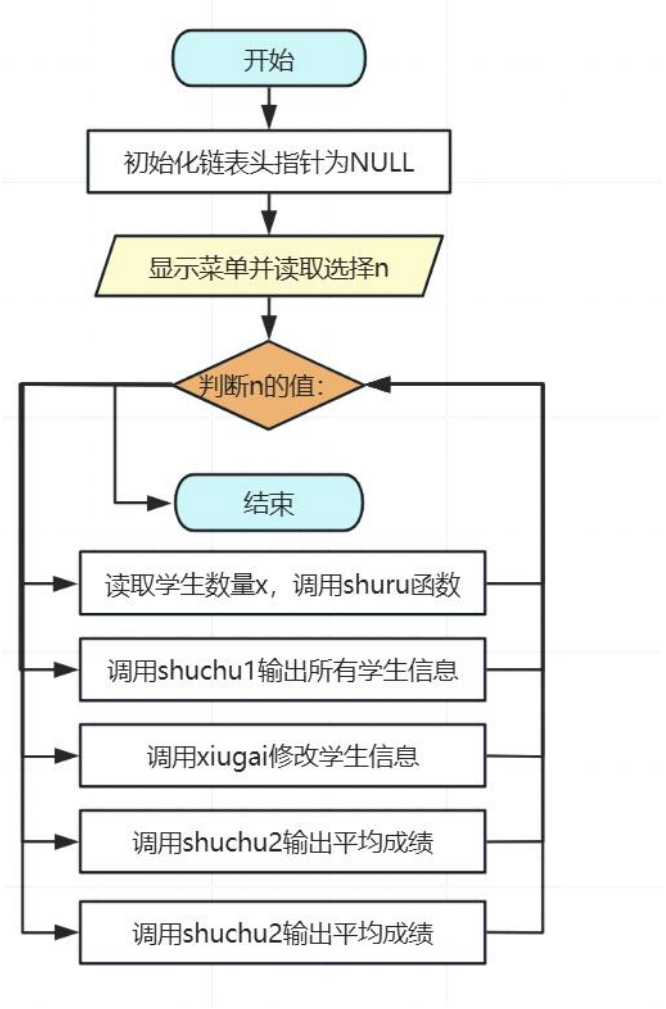


图 2-6 程序设计题 2 的流程图

程序源代码：

```
#include<stdio.h>
struct bits{
    unsigned int bits0:1;
    unsigned int bits1:1;
    unsigned int bits2:1;
    unsigned int bits3:1;
    unsigned int bits4:1;
    unsigned int bits5:1;
    unsigned int bits6:1;
    unsigned int bits7:1;
};
void f0(int b){
    printf("the function %d is called!\n",b-1);
}
void f1(int b){
```

```
16.      printf("the function %d is called!\n",b);
17.  }
18.  void f2(int b){
19.      printf("the function %d is called!\n",b+1);
20.  }
21.  void f3(int b){
22.      printf("the function %d is called!\n",b+2);
23.  }
24.  void f4(int b){
25.      printf("the function %d is called!\n",b+3);
26.  }
27.  void f5(int b){
28.      printf("the function %d is called!\n",b+4);
29.  }
30.  void f6(int b){
31.      printf("the function %d is called!\n",b+5);
32.  }
33.  void f7(int b){
34.      printf("the function %d is called!\n",b+6);
35.  }
36.  int main() {
37.      int x;
38.      scanf("%d",&x);
39.      void (*hanshu[8])(int) = {f0, f1, f2, f3, f4, f5, f6,
    f7};
40.      union {
41.          unsigned char byte;
42.          struct bits bits;
43.      } data;
44.      data.byte = (unsigned char)x;
45.
46.      if (data.bits.bits0) hanshu[0](data.bits.bits0);
47.      if (data.bits.bits1) hanshu[1](data.bits.bits1);
48.      if (data.bits.bits2) hanshu[2](data.bits.bits2);
49.      if (data.bits.bits3) hanshu[3](data.bits.bits3);
50.      if (data.bits.bits4) hanshu[4](data.bits.bits4);
51.      if (data.bits.bits5) hanshu[5](data.bits.bits5);
52.      if (data.bits.bits6) hanshu[6](data.bits.bits6);
53.      if (data.bits.bits7) hanshu[7](data.bits.bits7);
54.
55.      return 0;
56.  }
```

运行结果：

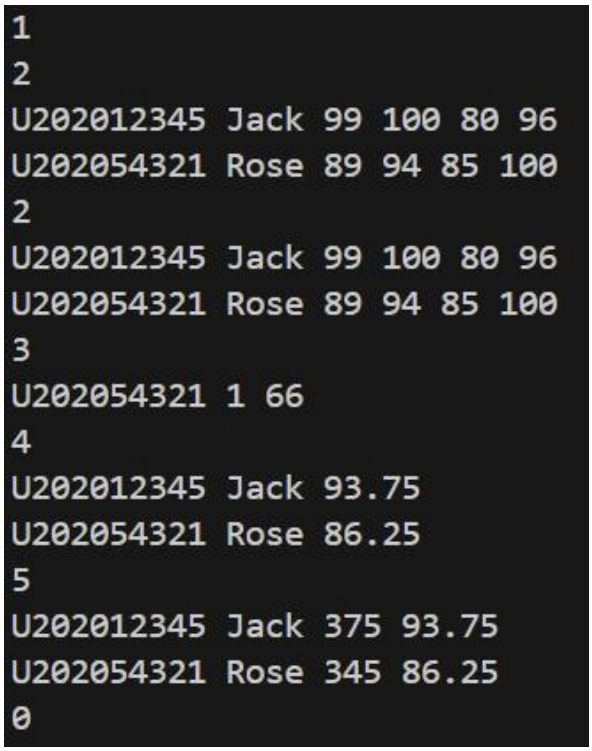


图 2-7 程序设计题 2 的程序运行结果示意图

3. **题目**

对程序设计题第（2）题的程序，增加按照平均成绩进行升序排序的函数，写出用交换结点数据域的方法升序排序的函数。排序可采用选择法或冒泡法。

解题思路：

1. 需要在原有学生成绩管理系统基础上增加按平均成绩升序排序功能
2. 采用冒泡排序算法，通过交换相邻节点的数据域实现排序
3. 比较相邻节点的总成绩（x1+x2+x3+x4），如果前节点大于后节点则交换
4. 交换两个节点的所有数据域（学号、姓名、四门成绩）

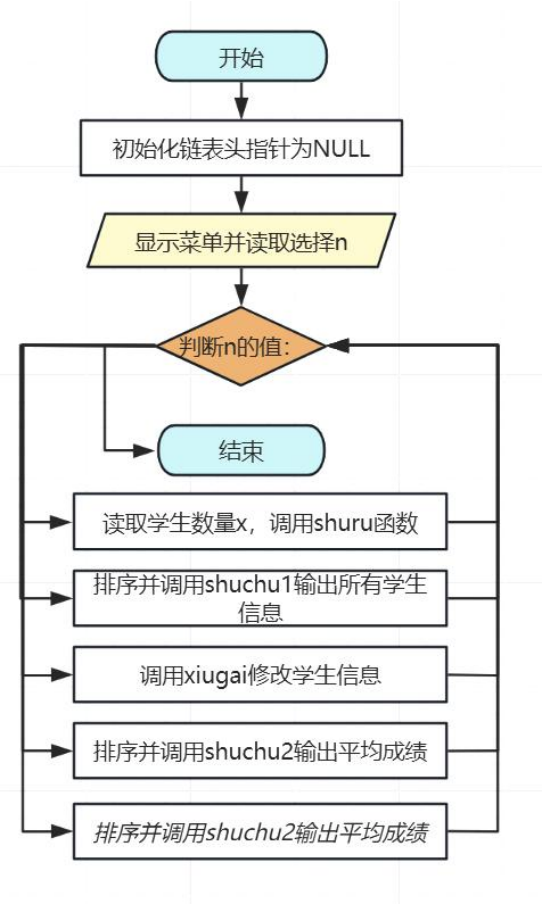


图 2-6 程序设计题 3 的流程图

程序源代码：

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
struct stu{
    char num[15];
    char name[10];
    int x1;
    int x2;
    int x3;
    int x4;
    struct stu*next;
};

void shuru(struct stu**biao,int n) {
    struct stu *p, *tail;
    int i;
    tail=*biao;
```

```
19.      if(tail!=NULL) {
20.          while (tail->next!=NULL){
21.              tail = tail->next;
22.          }
23.      }
24.      for (i=0;i<n;i++) {
25.          p=(struct stu*)malloc(sizeof(struct stu));
26.          scanf("%s %s %d %d %d %d",p->num,p->name,&p->x1,&
   p->x2,&p->x3,&p->x4);
27.          p->next=NULL;
28.          if (*biao==NULL) {
29.              *biao=p;
30.              tail=p;
31.          } else {
32.              tail->next=p;
33.              tail=p;
34.          }
35.      }
36.  }
37.
38.
39.  void shuchu1(struct stu*head){
40.  struct stu*p;
41.      for(p=head;p!=NULL;p=p->next){
42.          printf("%s %s %d %d %d %d\n",p->num,p->name,p->x1,
   p->x2,p->x3,p->x4);
43.      }
44.  }
45.
46.  void xiugai(struct stu**head){
47.  struct stu*p;
48.  char t[15];
49.  scanf(" %s ",t);
50.      for(p=*head;p!=NULL;p=p->next){
51.            if(strcmp((char*)p->num,t)==0){
52.              int q;
53.              scanf("%d",&q);
54.              switch(q){
55.                  case 1:
56.                  scanf("%d",&p->x1);
57.                  break;
58.                  case 2:
59.                  scanf("%d",&p->x2);
60.                  break;
```

```
                    case 3:
                    scanf("%d",&p->x3);
                    break;
                    case 4:
                    scanf("%d",&p->x4);
                    break;
                }
            }
    }
}

void paixu(struct stu*head){
struct stu*p,*tail;
char j1[15];
char j2[10];
int j3;
int j4;
int j5;
int j6;
for(tail=NULL;head!=tail;tail=p){
    for(p=head;p->next!=tail;p=p->next){
            if((p->x1+p->x2+p->x3+p->x4)>(p->next->x1+p->next->x2+p->next->x3+p->next->x4)){
             j3=p->x1;p->x1=p->next->x1;p->next->x1=j3;
             j4=p->x2;p->x2=p->next->x2;p->next->x2=j4;
             j5=p->x3;p->x3=p->next->x3;p->next->x3=j5;
             j6=p->x4;p->x4=p->next->x4;p->next->x4=j6;
                strcpy(j1,(char*)p->num);strcpy((char*)p->num,(char*)p->next->num);strcpy((char*)p->next->num,j1);
                strcpy(j2,(char*)p->name);strcpy((char*)p->name,(char*)p->next->name);strcpy((char*)p->next->name,j2);
            }
        }

}

}
void shuchu2(struct stu*head){
struct stu*p;
    for(p=head;p!=NULL;p=p->next){
        printf("%s %s %.2f\n",p->num,p->name,(p->x1+p->x2+p->x3+p->x4)/4.0);
    }
```

```
100. }
101.
102. void shuchu3(struct stu*head){
103. struct stu*p;
104.     for(p=head;p!=NULL;p=p->next){
105.         printf("%s %s %d %.2f\n",p->num,p->name,p->x1+p->
   x2+p->x3+p->x4,(p->x1+p->x2+p->x3+p->x4)/4.0);
106.     }
107. }
108.
109. int main(){
110.     struct stu *biao=NULL;
111.     int total=0;
112.     while(1){
113.         int n;int x;
114.         scanf("%d",&n);
115.         switch(n){
116.             case 0:
117.             return 0;
118.             case 1:
119.
120.             scanf("%d",&x);
121.             shuru(&biao,x);
122.             total+=x;
123.             break;
124.             case 2:
125.             paixu(biao);
126.             shuchu1(biao);
127.             break;
128.             case 3:
129.             xiugai(&biao);
130.             break;
131.             case 4:
132.             paixu(biao);
133.             shuchu2(biao);
134.             break;
135.             case 5:
136.             paixu(biao);
137.             shuchu3(biao);
138.             break;
139.         }
140.     }
141.     return 0;
142. }
```

运行结果：

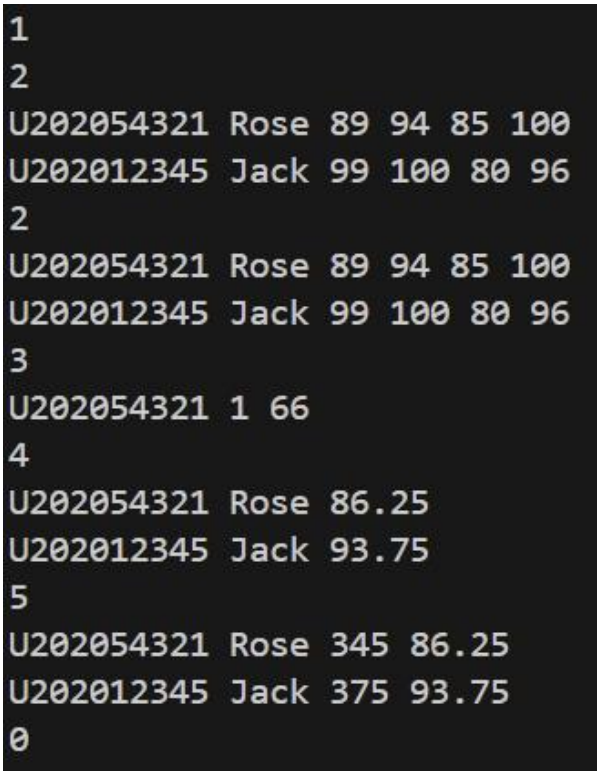


图 2-7 程序设计题 3 的程序运行结果示意图

4. **题目**

判断由键盘输入的任意长度字符串是否为“回文”。称正读和反读都相同的字符串为“回文”，例如“abba”和“abcba”是回文。

解题思路：

1. 链表创建函数 createLinkList：根据输入的字符串创建单向链表，每个节点存储一个字符，使用尾插法构建链表，保持字符顺序

2. 回文判断函数 judgePalindrome：使用快慢指针法找到链表中点，在寻找中点过程中反转前半部分链表，比较前半部分和后半部分的字符是否相同，根据比较结果输出判断结果

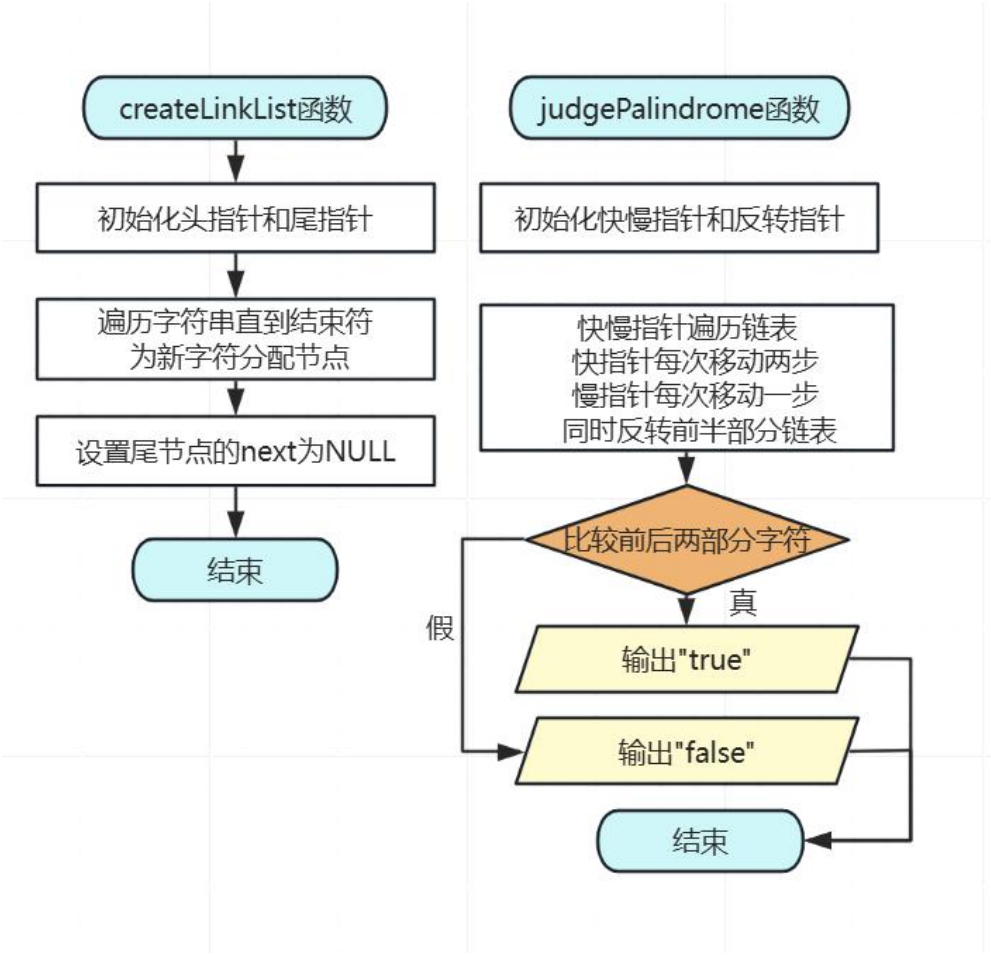


图 2-8 程序设计题 4 的流程图

程序源代码：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 定义结点类型
typedef struct c_node{
    char data;
    struct c_node *next;
} C_NODE;

// 创建链表函数
void createLinkList(C_NODE **headp, char s[])
{
    C_NODE *p, *tail = NULL;

    for (int i = 0; s[i] != '\0'; i++) {
        p = (C_NODE*)malloc(sizeof(C_NODE));
```

```
18.             p->data = s[i];
19.             p->next = NULL;
20.
21.             if (*headp == NULL) {
22.                 *headp = p;
23.             } else {
24.                 tail->next = p;
25.             }
26.             tail = p;
27.         }
28.     }
29.
30.     // 判断回文函数
31.     void judgePalindrome(C_NODE *head)
32.     {
33.         if (head == NULL) {
34.             printf("true");
35.             return;
36.         }
37.
38.         C_NODE *slow = head;
39.         C_NODE *fast = head;
40.         C_NODE *prev = NULL;
41.
42.         // 使用快慢指针找到中点并反转前半部分链表
43.         while (fast != NULL && fast->next != NULL) {
44.             fast = fast->next->next;
45.             C_NODE *next_slow = slow->next;
46.             slow->next = prev;
47.             prev = slow;
48.             slow = next_slow;
49.         }
50.
51.         C_NODE *first_half = prev;
52.         C_NODE *second_half = slow;
53.
54.         // 如果是奇数长度，跳过中间节点
55.         if (fast != NULL) {
56.             second_half = slow->next;
57.         }
58.
59.         // 比较前后两部分
60.         while (first_half != NULL && second_half != NULL) {
61.             if (first_half->data != second_half->data) {
```

```
62.                 printf("false");
63.                 return;
64.             }
65.             first_half = first_half->next;
66.             second_half = second_half->next;
67.         }
68.
69.         printf("true");
70.     }
71.
72.     // 释放链表内存
73.     void freeList(C_NODE *head) {
74.         C_NODE *current = head;
75.         while (current != NULL) {
76.             C_NODE *next = current->next;
77.             free(current);
78.             current = next;
79.         }
80.     }
81.
82.     // 主函数
83.     int main() {
84.         char input[100];
85.         printf("请输入一个字符串: ");
86.         scanf("%s", input);
87.
88.         C_NODE *head = NULL;
89.         createLinkList(&head, input);
90.
91.         judgePalindrome(head);
92.
93.         freeList(head);
94.
95.         return 0;
96.     }
```

运行结果：

```
请输入一个字符串: aaabbaaa
true
```

图 2-9 程序设计题 4 的程序运行结果示意图

5. 题目

对程序设计题第（3）题进一步写出用交换结点指针域的方法升序排序的函数。

解题思路：

1. 使用冒泡排序算法，但通过调整节点指针域而非数据域实现排序， 创建临时头节点简化边界处理通过改变节点间的连接关系实现排序，避免大量数据拷贝

2. 初始化空链表，进入主循环，提供菜单界面，根据用户选择调用相应功能，在输出成绩前自动进行排序，确保结果有序

3. 排序实现细节， 使用双重循环遍历链表， 比较相邻节点的总成绩，通过调整节点指针域交换节点位置，使用临时头节点简化链表头部处理

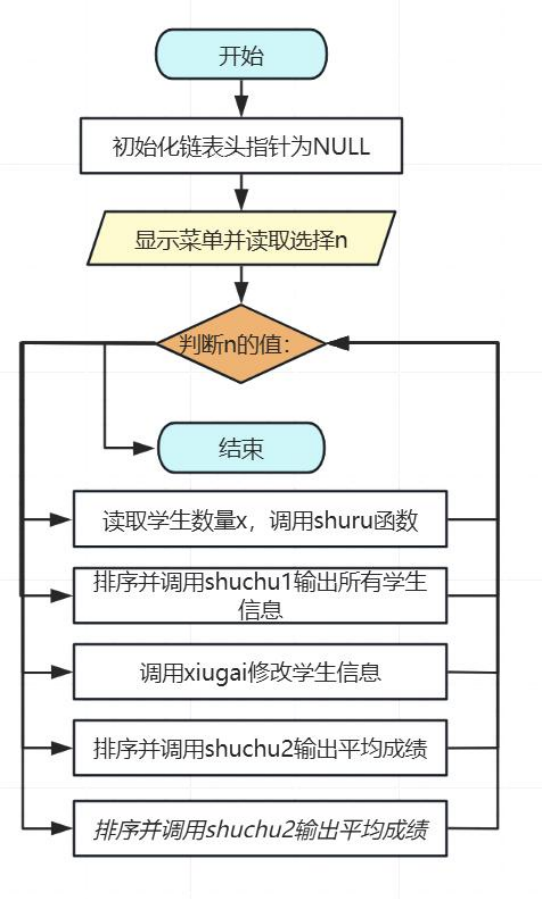


图 2-10 程序设计题 5 的流程图

程序源代码：

```
1.  #include<stdio.h>
2.  #include<stdlib.h>
3.  #include<string.h>
```

```
4.  struct stu{
5.      char num[15];
6.      char name[10];
7.      int x1;
8.      int x2;
9.      int x3;
10.     int x4;
11.     struct stu*next;
12. };
13.
14.
15. void shuru(struct stu**biao,int n) {
16.     struct stu *p, *tail;
17.     int i;
18.     tail=*biao;
19.     if(tail!=NULL) {
20.         while (tail->next!=NULL){
21.             tail = tail->next;
22.         }
23.     }
24.     for (i=0;i<n;i++) {
25.         p=(struct stu*)malloc(sizeof(struct stu));
26.          scanf("%s %s %d %d %d %d",p->num,p->name,&p->x1,&
    p->x2,&p->x3,&p->x4);
27.         p->next=NULL;
28.         if (*biao==NULL) {
29.             *biao=p;
30.             tail=p;
31.         } else {
32.             tail->next=p;
33.             tail=p;
34.         }
35.     }
36. }
37.
38.
39. void shuchu1(struct stu*head){
40. struct stu*p;
41.     for(p=head;p!=NULL;p=p->next){
42.         printf("%s %s %d %d %d %d\n",p->num,p->name,p->x1,
    p->x2,p->x3,p->x4);
43.     }
44. }
45.
```

```
46.  void xiugai(struct stu**head){
47.  struct stu*p;
48.  char t[15];
49.  scanf(" %s ",t);
50.      for(p=*head;p!=NULL;p=p->next){
51.            if(strcmp((char*)p->num,t)==0){
52.              int q;
53.              scanf("%d",&q);
54.              switch(q){
55.                  case 1:
56.                  scanf("%d",&p->x1);
57.                  break;
58.                  case 2:
59.                  scanf("%d",&p->x2);
60.                  break;
61.                  case 3:
62.                  scanf("%d",&p->x3);
63.                  break;
64.                  case 4:
65.                  scanf("%d",&p->x4);
66.                  break;
67.              }
68.            }
69.      }
70.  }
71.
72.  void paixu(struct stu**head){
73.  struct stu*p,*tail,*last,*toujiedian;
74.  toujiedian=(struct stu*)malloc(sizeof(struct stu));
75.  toujiedian->next=*head;
76.  for(tail=NULL;toujiedian->next!=tail;tail=p){
77.       for(last=toujiedian,p=toujiedian->next;p->next!=tail;
   last=p,p=p->next){
78.              if((p->x1+p->x2+p->x3+p->x4)>(p->next->x1+p->
   next->x2+p->next->x3+p->next->x4)){
79.                  last->next=p->next;
80.                  p->next=p->next->next;
81.                  last->next->next=p;
82.                  p=last->next;
83.              }
84.          }
85.
86.  }
87.   *head=toujiedian->next;
```

```
88.  }
89.  void shuchu2(struct stu*head){
90.  struct stu*p;
91.      for(p=head;p!=NULL;p=p->next){
92.          printf("%s %s %.2f\n",p->num,p->name,(p->x1+p->x2
   +p->x3+p->x4)/4.0);
93.      }
94.  }
95.
96.  void shuchu3(struct stu*head){
97.  struct stu*p;
98.      for(p=head;p!=NULL;p=p->next){
99.          printf("%s %s %d %.2f\n",p->num,p->name,p->x1+p->
   x2+p->x3+p->x4,(p->x1+p->x2+p->x3+p->x4)/4.0);
100.     }
101. }
102.
103. int main(){
104.     struct stu *biao=NULL;
105.     int total=0;
106.     while(1){
107.         int n;int x;
108.         scanf("%d",&n);
109.         switch(n){
110.             case 0:
111.             return 0;
112.             case 1:
113.
114.             scanf("%d",&x);
115.             shuru(&biao,x);
116.             total+=x;
117.             break;
118.             case 2:
119.             paixu(&biao);
120.             shuchu1(biao);
121.             break;
122.             case 3:
123.             xiugai(&biao);
124.             break;
125.             case 4:
126.             paixu(&biao);
127.             shuchu2(biao);
128.             break;
129.             case 5:
```

```
130.                paixu(&biao);
131.                shuchu3(biao);
132.                break;
133.            }
134.        }
135.        return 0;
136. }
```

运行结果：

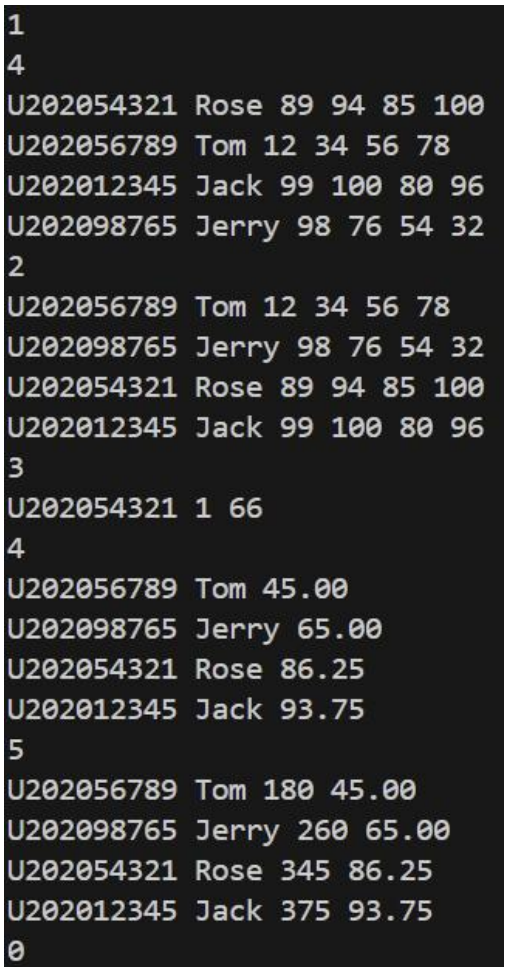


图 2-11 程序设计题 5 的程序运行结果示意图

5. **题目**

利用值栈对逆波兰表达式进行求值。逆波兰表达式从键盘输入，其中的运算符仅包含加、减、乘、除 4 种运算，表达式中的数都是十进制数，用换行符结束输入。由于逆波兰表达式的长度不限，所以值栈要用后进先出链表实现。

解题思路：

1. 使用栈来处理逆波兰表达式，遇到操作数时压入栈中，遇到运算符时从栈

中弹出两个操作数进行计算，将计算结果压回栈中，最终栈中剩下的唯一元素即为表达式结果

2. 使用单向链表实现栈，每个节点包含整数数据和指向下一节点的指针，通过头插法实现栈的 push 操作，通过头节点删除实现栈的 pop 操作

3. 逐个读取 token 直到遇到换行符， 判断 token 是运算符还是操作数，操作数转换为整数后入栈，运算符触发相应的计算操作

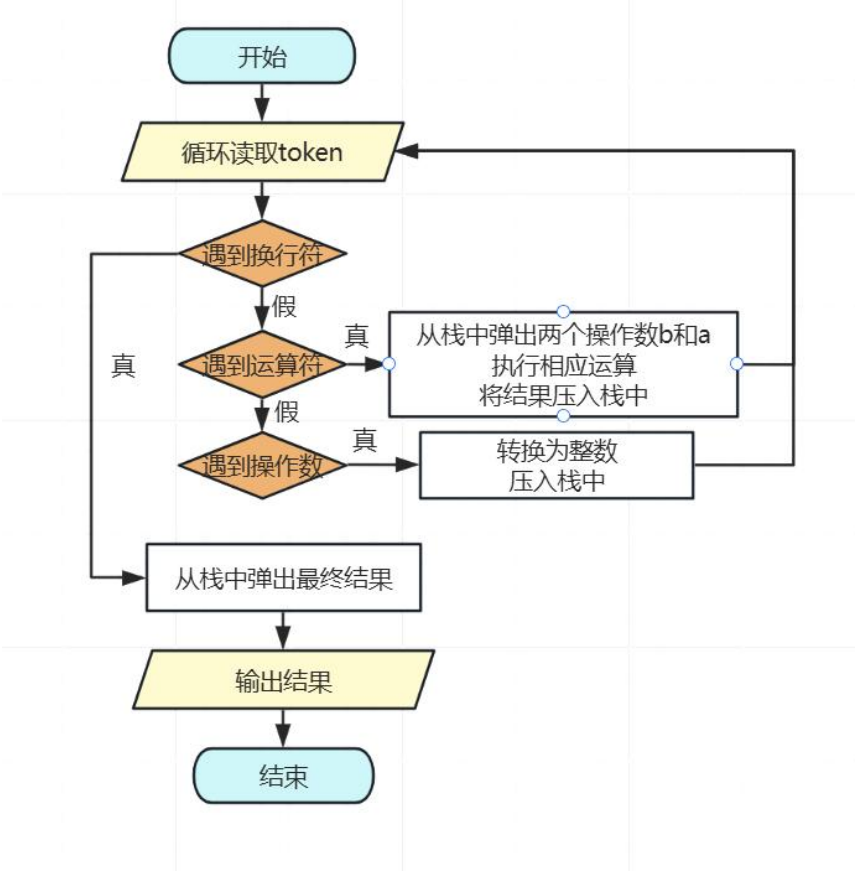


图 2-12 程序设计题 6 的流程图

程序源代码：

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

```

```
11.  void push(Node** top, int value);
12.  int pop(Node** top);
13.  int isOperator(const char* token);
14.  int performOperation(int a, int b, char op);
15.
16.  int main() {
17.      Node* stack = NULL;
18.      char token[100];
19.      while (scanf("%99s", token) == 1) {
20.          if (token[0] == '\n') break;
21.          if (isOperator(token)) {
22.              int b = pop(&stack);
23.              int a = pop(&stack);
24.              int result = performOperation(a, b, token[0]);
25.              push(&stack, result);
26.          } else {
27.              int num = atoi(token);
28.              push(&stack, num);
29.          }
30.      }
31.
32.
33.      int result = pop(&stack);
34.      printf("%d\n", result);
35.
36.      return 0;
37.  }
38.
39.
40.  void push(Node** top, int value) {
41.      Node* newNode = (Node*)malloc(sizeof(Node));
42.      newNode->data = value;
43.      newNode->next = *top;
44.      *top = newNode;
45.  }
46.
47.  int pop(Node** top) {
48.      if (*top == NULL) {
49.          exit(EXIT_FAILURE);
50.      }
51.      Node* temp = *top;
52.      int value = temp->data;
53.      *top = (*top)->next;
```

```
54.      free(temp);
55.      return value;
56.  }
57.
58.  int isOperator(const char* token) {
59.      return (token[0]=='+'||token[0]=='-'||token[0]=='*'||
   token[0]=='/') && token[1] == '\0';
60.  }
61.
62.  int performOperation(int a, int b, char op) {
63.      switch (op) {
64.          case '+': return a + b;
65.          case '-': return a - b;
66.          case '*': return a * b;
67.          case '/':
68.              if (b == 0) {
69.                  fprintf(stderr, "除零错误\n");
70.                  exit(EXIT_FAILURE);
71.              }
72.              return a / b;
73.          default:
74.              fprintf(stderr, "无效运算符: %c\n", op);
75.              exit(EXIT_FAILURE);
76.      }
77.  }
```

运行结果：

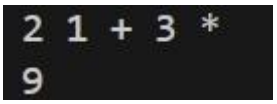


图 2-13 程序设计题 6 的程序运行结果示意图

2.4 小结

本次实验主要考察对各种操作符优先级，运算顺序，结构体和链表的使用的理解。通过实验，我熟悉了结构的说明和引用、结构的指针、结构数组、以及函数中使用结构的方法。了解动态储存分配函数的用法，掌握自引用结构，单向链表的创建、遍历、结点的增删、查找等操作。实验过程中，我也遇到了一些问题，以下是我的经验和总结。

善用调试：在程序改错题中，我没有发现传入的是一级指针，函数形参的改变不会影响原指针的值，通过调试，我发现了主函数中 p 是个空指针，因此程序没有任何输出，从而发现了问题所在。

认识到结构体的重要性：在编写学生管理系统时，我认识到结构体能够将多个不同类型的数据封装成一个整体，这对于管理和操作复杂数据（如学生的学号、姓名、成绩等）至关重要，是构建链表等数据结构的基础。

链表的逻辑性：通过实践，我深刻体会到链表是一种逻辑上连续而物理上非连续的数据结构。每一个节点都通过指针指向下一个节点，这种通过指针连接形成的“链”是链表的核心。在插入和删除节点时，必须清晰地理解并维护好节点之间前驱和后继的逻辑关系，否则就会导致链表的断裂或内存问题。

运算优先级：通过第一道题，我总结出，同一优先级的运算符，运算次序由结合方向所决定。简单记就是：>> 算术运算符 >> 关系运算符 >&&>||>>&&>||> 赋值运算符。

大型程序编写：程序设计题中，我们需要做一个具有六个功能的学生管理系统。这既需要我们编写多个函数实现相应功能，还需要我们设计格式，提示语，注意各个函数中链表的内存分配，排序等是否存在问题，这对提升我们的编程能力十分重要。

参考文献

[1]卢萍,李开,王多强，甘早斌. C 语言程序设计典型题解与实验指导,北京：清华大学出版社,2019

[2]卢萍,李开,王多强，甘早斌. C 语言程序设计,北京：清华大学出版社,2019